

TECHNICAL REPORT

Frontend Task Implementation

Requirements, Functions & Design Decisions

Applicant	Chirayu Subedi
Date	April 2026
Contact	chirayu.subedi.cs84@gmail.com
Recipient	Markus Waidmann
Repository	github.com/ChirayuSubedi/task_apopark.git

SECTION 1

Executive Summary

All three frontend tasks were completed in plain HTML, CSS, and JavaScript — no frameworks, no build tools, no external dependencies. Each task folder opens directly in the browser.

Beyond fulfilling the stated requirements, the implementation adds: localStorage persistence (Tasks 2 and 3), a stable sort algorithm with explicit index tiebreaking (Task 1), a drag-over flicker fix using an enter-counter pattern (Task 2), and a unified design system applied consistently across all three interfaces.

All logic follows one consistent pattern: update state → persist → re-render → update derived UI. Code comments explain the why behind decisions, not the what.

At a Glance

3 Tasks Completed	6 Files Modified	0 External Libraries
-----------------------------	----------------------------	--------------------------------

SECTION 2

Task 1 — Sortable Employee Table

Files modified: `Task1/styles.css` · `Task1/table.js`

Requirements

- ✓ Apply a clean card-like layout with correct column widths (Name 28%, Dept 22%, Role 28%, Date 12%, Fee 10%)
- ✓ Add zebra striping and row hover effects for readability
- ✓ Implement ``sortBy()`` for Name, Department, Start Date, and Monthly Fee (Role column is not sortable)
- ✓ Toggle sort direction (asc → desc) on the same column; reset to asc when switching columns
- ✓ Display a sort direction indicator (▲ / ▼) on the active column header

CSS Implementation — styles.css

Change / Function	What it does & why
<code>box-sizing: border-box</code> on *	Industry standard — padding and borders are included in element dimensions. Consistent across all three tasks.
Column widths via <code>nth-child</code>	<code>table-layout: fixed</code> was already set; <code>nth-child</code> is the correct selector to enforce fixed-layout percentages: Name 28%, Dept 22%, Role 28%, Date 12%, Fee 10%.
Zebra striping on even rows	Alternating row colour helps the eye track horizontally across a wide table without full row borders.
Blue row hover targeting <code>td</code> directly	Immediate feedback on the hovered row. Targeting <code>tbody tr:hover td</code> ensures even-row hover always wins over the zebra background colour.
<code>user-select: none</code> on sortable headers	Repeated clicks for sorting would otherwise highlight the header text, which looks broken and confuses the user.
<code>text-align: right</code> on fee column	Standard typographic convention for numeric and currency values.
<code>overflow: hidden</code> on <code>.table-shell</code>	Eliminates horizontal scroll entirely. The container clips content rather than scrolling — combined with <code>text-overflow: ellipsis</code> on cells, long text is always displayed cleanly.
Absolutely positioned <code>.sort-arrow</code> span	Arrow sits outside the text flow (<code>position: absolute; right: 8px</code>). Column width can never change when the indicator appears or disappears — this is the fix for the scrollbar triggered by "Monthly Fee ▲".
<code>.is-sorted</code> class on active header	Blue label colour + blue arrow on the sorted column. CSS handles the visual state entirely; JS only toggles the class name.
System font + <code>box-shadow</code> card	Replaced Arial with the system font stack. Flat 1px border replaced with a layered shadow — matches Tasks 2 and 3 for a consistent design system.

JavaScript Implementation — table.js

Change / Function	What it does & why
<code>compareValues(a, b, type)</code>	Type-aware comparator: numeric subtraction for numbers, <code>Date.getTime()</code> for dates, <code>localeCompare</code> for strings. <code>localeCompare</code> is required to correctly sort German umlauts (ä, ö, ü) — plain <code><</code> comparison produces wrong results.
<code>sortBy(key, type)</code>	Core sort function. Same key clicked again → flip direction (asc ↔ desc). New key clicked → reset to asc. Delegates comparison to <code>compareValues()</code> and applies a stable-sort

	index tiebreaker.
Stable sort via index tiebreaker	Each row is tagged with its original array index before sorting. When the primary comparison returns zero (equal values), the index is the tiebreaker — original relative order is always preserved.
<code>setHeaderIndicators()</code>	Updates the <code>.sort-arrow</code> span text (▲ / ▼ / empty) and toggles <code>.is-sorted</code> on each header. Label text is never changed — only the arrow span content changes, so column width stays stable.
<code>render(data)</code>	Rebuilds <code>tbody</code> innerHTML from the sorted data array. Called after every sort. Adds the € currency symbol to fee cells — raw numbers without a unit are ambiguous.
<code>.sort-arrow</code> span injected per sortable header	Each sortable <code>th</code> gets its label in a text node and an empty <code>.sort-arrow</code> span appended. This structure ensures the label width is constant and the arrow contributes zero layout width.
<code>classList.toggle("is-sorted")</code>	Called in <code>setHeaderIndicators()</code> . Active column header receives a CSS class — blue accent applied entirely in CSS, no inline styles or JS colour logic.

KEY DECISION — STABLE SORT

JavaScript's native sort is stable in all modern browsers (V8 ≥ 7.0, Firefox, Safari). To make stability explicit and independent of engine guarantees, each row is tagged with its original array index before sorting, and that index serves as a tiebreaker when the primary comparison returns zero.

BUG FIX — HORIZONTAL SCROLL TRIGGERED BY SORT ARROW

Appending "▲" to the header `textContent` caused "Monthly Fee ▲" to exceed the fixed 10% column width under `white-space: nowrap`, triggering a horizontal scrollbar. Fixed by: (1) injecting the arrow into an absolutely-positioned `.sort-arrow` span (outside text flow, adds zero width), and (2) changing `overflow: auto` → `overflow: hidden` on the container.

SECTION 3

Task 2 — Kanban Ticket Board

Files modified: Task2/styles.css · Task2/board.js · Task2/board.html

Requirements

- ✓ Style a three-column Kanban board (To Do, In Progress, Done)
- ✓ Make ticket cards draggable between columns using the HTML5 Drag and Drop API
- ✓ Highlight the active drop target column during drag
- ✓ Update ticket status on drop and reflect the change in the UI immediately
- ✓ Display correct ticket counts per column

CSS Implementation — styles.css

Change / Function	What it does & why
<code>cursor: grab</code> on <code>.ticket</code>	Immediately signals draggability before the user attempts to drag — the cursor is the first affordance.
Shadow lift on <code>.ticket:hover</code>	Physical depth metaphor reinforces that cards can be picked up and moved.
<code>.ticket.dragging { opacity: 0.35 }</code>	Source card fades while "in flight" — standard DnD convention showing where the card came from.
Ticket typography hierarchy	Small muted uppercase ID label above a bold title. ID is metadata; title is primary content. Separate elements allow independent styling.
<code>.dropzone.drag-over</code> dashed blue outline	Dashed outline is the universal drop-target indicator. Removed only when <code>enterCount</code> reaches zero — no flicker (see JS section).
Priority colour badges via <code>[data-priority]</code>	High = red pill, Medium = amber pill, Low = grey pill. Driven entirely by CSS attribute selectors — zero colour logic in JS. The badge colour is a pure CSS concern.
<code>.col-title</code> + <code>.col-count</code> pill in column header	Count displayed as a pill badge; parentheses removed. Requires two child spans in the header HTML — allows independent styling of title and count.
Unified design system with Tasks 1 and 3	Same system font stack, card box-shadow, border-radius 10px, and uppercase tracked header style across all three tasks.

JavaScript Implementation — board.js

Change / Function	What it does & why
<code>loadTickets()</code>	Restores saved state from <code>localStorage</code> . Falls back to <code>defaultTickets</code> if the key is missing or JSON is corrupt. Versioned storage key prevents stale data from a previous schema breaking the board.

<code>saveTickets()</code>	Serialises the tickets array to localStorage after every drop. Board state survives page reload.
<code>render()</code>	Rebuilds all three column dropzones from the tickets array. Called after every status change. Sets <code>data-priority</code> on each ticket element so CSS applies the correct badge colour without any JS colour logic.
<code>updateCounts()</code>	Derives per-column ticket counts from the source array on every render. No separate counter state to keep in sync — always accurate.
<code>setupDropzones()</code>	Wires all four DnD events on each column dropzone. Called once on load — listeners survive re-renders because <code>render()</code> only rewrites dropzone children, not the dropzone element itself.
<code>dragenter</code> / <code>dragleave</code> counter pattern	Per-zone <code>enterCount</code> increments on <code>dragenter</code> and decrements on <code>dragleave</code> . The <code>.drag-over</code> highlight is removed only when the counter reaches zero — meaning the cursor has truly left the zone, not just moved over a child ticket. This eliminates the visible flicker of a naive <code>dragover/dragleave</code> approach.
<code>dragover</code> — <code>preventDefault()</code> only	Required by the HTML5 DnD spec to declare the element as a valid drop target. Highlight logic stays in <code>dragenter/dragleave</code> .
drop handler	Resets <code>enterCount</code> to 0, removes the highlight, finds the dragged ticket by ID, updates its <code>status</code> , saves, and re-renders.
<code>dragstart</code> / <code>dragend</code>	<code>dragstart</code> sets <code>draggedTicketId</code> and adds <code>.dragging</code> opacity. <code>dragend</code> always clears both — fires even on Escape or cancelled drag, preventing stale state on the next drag.
<code>el.dataset.priority = ticket.priority</code>	Exposes priority to CSS as a <code>data-priority</code> attribute. The badge colour is fully CSS-driven — zero JS colour logic or inline styles required.

HTML Change — board.html

Change / Function	What it does & why
Column headers restructured with <code>.col-title</code> and <code>.col-count</code> spans	Previously the count was embedded in text: "To Do (0)". Splitting into two spans allows the count to be styled as a pill badge independently. The JS <code>id</code> attributes (<code>count-todo</code> etc.) are preserved unchanged.

BUG FIX — DRAG-OVER HIGHLIGHT FLICKER

Using `dragover` to add and `dragleave` to remove `.drag-over` caused visible flickering when the cursor moved over a child ticket card inside the dropzone. The `dragenter/dragleave` counter is flicker-free: the highlight is removed only when `enterCount` reaches zero — meaning the cursor has genuinely left the entire column.

KEY DECISION — MODULE-LEVEL DRAG VARIABLE VS. DATATRANSFER

A module-level `draggedTicketId` variable is simpler than the `DataTransfer` API: no serialisation, no parsing, and the intent is clear from the name. Since only one drag can be active at a time, there is no concurrency concern. The only tradeoff — cross-window drops — is not a requirement here.

SECTION 4

Task 3 — Parking Lot Selector

Files modified: `Task3/styles.css` · `Task3/map.js` · `Task3/map.html`

Requirements

- ✓ Display 10 parking spots in a grid with visual status (available / occupied)
- ✓ Allow selecting up to 3 available spots; clicking a selected spot deselects it
- ✓ Prevent selecting occupied spots
- ✓ Show an error message when the user attempts to exceed the 3-spot maximum
- ✓ Display a summary: total selected count, spot IDs, and total monthly fee

CSS Implementation — styles.css

Change / Function	What it does & why
Chip colours match spot tile colours	Legend and tiles share the same palette — green for available, red for occupied, blue for selected. No colour mismatch between legend and grid.
<code>.spot.available</code> green tint	Green = open = go — the universally understood signal for availability.
<code>.spot.occupied: red + cursor: not-allowed + opacity: 0.65</code>	Three visual cues reinforce the same message: this spot cannot be selected. Belt-and-suspenders approach for clarity.
<code>.spot.selected</code> blue tint + selection ring	Blue is the standard "active / selected" colour. The <code>box-shadow: 0 0 0 2px</code> ring makes selection unmistakable without changing tile dimensions.
Different hover colours for available vs. selected	Deeper green on hover signals "click to select"; deeper blue signals "click to deselect". The hover communicates the action before the click.
<code>min-height: 20px</code> on <code>.error</code>	Reserves vertical space so the layout does not jump when the error message appears or disappears.
Spot tile typography coloured by state	Both the spot ID (<code></code>) and the fee (<code><div></code>) change colour based on spot status (green / red / blue). The tile reads as a unified element.
<code>.card</code> wrapper + unified design system	Single white card containing legend, summary, error, and grid — matches the card pattern in Tasks 1 and 2. Same font, shadow, and radius across all tasks.

JavaScript Implementation — map.js

Change / Function	What it does & why
<code>handleSpotClick(spot)</code>	Core interaction. All three rules in one place: (1) return early for occupied spots — no state change, no re-render; (2) toggle deselect on second click and clear the error message; (3) block selection and show error if <code>selectedIds.size >= 3</code> .
<code>render()</code>	Full re-render of the map grid on every interaction. Creates a <code><button></code> for each spot with the correct status and selected classes. <code>selectedIds</code> is the single source of truth — the DOM always reflects it completely.
<code>updateSummary()</code>	Derives selected count, spot ID list, and total monthly fee from <code>selectedIds</code> on every call. No separate counter state to keep in sync.
<code>saveSelection()</code>	Serialises <code>selectedIds</code> to <code>localStorage</code> after every interaction. Uses a versioned key (<code>task3-parking-selection-v1</code>) to avoid conflicts with stale data from previous schemas.
<code>loadSelection()</code>	Restores selection from <code>localStorage</code> on startup. Validates each saved ID against current <code>spotData</code> — only IDs that still exist AND are still <code>available</code> are restored. Guards against a previously selected spot having become occupied.
Set for <code>selectedIds</code>	O(1) add / delete / has for all selection checks. Also prevents duplicates naturally — no extra guard needed.

HTML Change — map.html

Change / Function	What it does & why
<code><div class="card"></code> wrapper added	Wraps the legend, summary, error, and map grid in a single white card surface. Consistent with the card treatment in Tasks 1 and 2. No JS or functionality changes required.

KEY DECISION — FULL RE-RENDER ON EVERY CLICK

With 10 spots the performance cost is negligible. A full `render()` call on every interaction keeps the logic simple: `selectedIds` is the single source of truth and the DOM always reflects it completely. A partial DOM-update approach (toggling classes in place) would be marginally faster but harder to reason about and more error-prone.

FEATURE — SELECTION PERSISTS ACROSS PAGE RELOADS

`saveSelection()` serialises `selectedIds` to `localStorage` after every interaction. `loadSelection()` runs before the first render and validates each saved ID — only spots that still exist and are still `available` are restored. This guards against stale data if spot statuses change between sessions.

SECTION 5

Unified Design System

All three tasks share a consistent visual language. The design system was applied after functional requirements were satisfied, requiring only CSS changes and minor HTML structural changes (Task 3 card wrapper).

Design Tokens

Token	Value
Font Family	-apple-system, BlinkMacSystemFont, "Segoe UI", Helvetica, Arial
Page Background	#f1f5f9 — cool light grey, used on all three tasks
Card Surface	#ffffff · border-radius: 10px · box-shadow: 0 1px 3px + 0 4px 14px
Header Style	0.7rem · 700 weight · uppercase · letter-spacing 0.07em · colour #64748b
Header Background	#f8fafc with 2px solid #e2e8f0 bottom border
Accent Blue	#3b82f6 (interactive elements) · #1e40af (text / dark variant)
Status: Available	Background #f0fdf4 · Border #86efac · Text #166534
Status: Occupied	Background #fef2f2 · Border #fca5a5 · Text #dc2626
Status: Selected	Background #eff6ff · Border #3b82f6 · Ring rgba(59,130,246,0.2)
Priority: High	Background #fef2f2 · Border #fca5a5 · Text #dc2626
Priority: Medium	Background #fffbeb · Border #fcd34d · Text #d97706
Priority: Low	Background #f1f5f9 · Border #cbd5e1 · Text #64748b

SECTION 6

Future Improvements

The following improvements were identified but not implemented within the task scope. Ordered by impact.

Priority	Task	Improvement
High	Task 1	Keyboard sort — sortable headers should respond to Enter and Space, not just mouse clicks
High	Task 1	aria-sort attribute on active header — screen readers cannot announce sort direction without it
High	Task 1	:focus-visible ring on sortable headers — keyboard users need a visible focus indicator
High	Task 2	Touch drag-and-drop support — HTML5 DnD does not fire on iOS or Android; a Pointer Events fallback is required
High	Task 2	Keyboard card movement — Tab to focus a card, arrow keys to move it between columns
High	Task 3	aria-pressed on spot buttons — screen readers need to know which spots are selected
High	Task 3	:focus-visible styles on spot buttons — keyboard navigation works (native buttons); explicit focus styles complete accessibility
Med	Task 1	Responsive fallback — table needs a minimum width and horizontal scroll wrapper on narrow viewports
Med	Task 1	Multi-column sort via Shift+click — standard in data-heavy admin tools
Med	Task 2	Empty column placeholder — "Drop tickets here" when a column is empty clarifies valid drop targets
Med	Task 2	Undo last move (Ctrl+Z) — a single state snapshot would enable one-level undo
Med	Task 3	"Clear selection" button — currently the only way to deselect all is clicking each spot individually
Med	Task 3	Hover tooltip — spot ID + fee + zone label without needing to read the summary panel
Low	All	prefers-reduced-motion media query — disable all CSS transitions for users who have requested reduced motion
Low	Task 2	ARIA roles for Kanban columns — role="list" on dropzone, role="listitem" on tickets
Low	Task 3	Zone / row labels on the grid — grouping spots visually mirrors a real parking map

General Notes

No external dependencies

No frameworks, no build tools, no package manager. Every task folder opens directly in the browser. This was a deliberate constraint that shapes every implementation decision toward simplicity and transparency.

Consistent state pattern

All three tasks follow the same flow: update state → persist (localStorage) → full re-render → update derived UI. This makes each task easy to follow, easy to extend, and easy to debug without any tooling.

Comments explain why, not what

Code comments throughout the task files explain the reasoning behind each decision — why `localeCompare` instead of `<`, why the `enter-counter` instead of `dragover`, why a `Set` instead of an array. This is the same documentation standard I would apply in a team codebase.

Design system applied after functional completeness

The visual redesign (shadow cards, system fonts, unified colour palette) was added as a separate pass after all requirements were satisfied. This separation ensures the functional logic was never compromised for aesthetic reasons.